



Dart

¿Qué es Dart?

Dart es un lenguaje de programación desarrollado por Google que se utiliza para construir aplicaciones web, móviles y de servidor. Es conocido por su rendimiento y su capacidad para crear interfaces de usuario atractivas y fluidas.

Hola, Mundo en Dart:

Primero comenzaremos con lo clásico el cual es un hola mundo.

Cabe mencionar que primero se pondrá la sintaxis y luego la explicación por partes.

Sintaxis:

```
void main(){  
    print('Hola Mundo');  
}
```

void main(): Esta línea define la función principal del programa. En Dart, todo programa comienza su ejecución desde la función **main**. La palabra clave **void** indica que esta función no retorna ningún valor.

{ } (Llaves): Delimitan el bloque de código asociado a la función **main**. Todas las instrucciones que pertenecen a la función principal están dentro de estas llaves.

print('Hola Mundo'); Esta es la instrucción que imprime el mensaje "Hola, Mundo!" en la consola. La función **print** es utilizada para enviar información a la salida estándar, que en este caso es la consola. El mensaje 'Hola, Mundo!' está encerrado entre comillas simples, indicando que es una cadena de texto.

Entonces, en resumen, el programa completo consta de una función principal (**main**) que imprime el mensaje 'Hola, Mundo!' en la consola. Cuando ejecutas este programa, verás la salida en la consola que muestra el saludo.

Para ejecutar este programa, puedes guardarlo en un archivo con extensión **.dart** (por ejemplo, **hola_mundo.dart**) y luego ejecutarlo desde la línea de comandos con el siguiente comando:

```
PS C:\Users\Joshu\Documents\DartCurso\dart_basics\bin\Leccion1> dart .\hola_mundo.dart  
Hola Mundo
```

Lección 1: Comentarios

```
void main(){  
    // Esto es un comentario de una línea  
  
    /*  
    Esto es un comentario de  
    varias líneas  
    fin  
    */  
  
    /// Este es un comentario de documentacion  
    /// se usa para documentar funciones y clases  
  
    // Imprimir saludo  
    print('Hola');  
}
```

Los comentarios en Dart, al igual que en otros lenguajes de programación, son una herramienta esencial para:

- Mejorar la legibilidad del código
- Explican la lógica detrás de funciones, clases o bloques complejos.
- Hacen el código más comprensible para otros desarrolladores (o para ti en el futuro).
- Facilitar el trabajo en equipo
- Permiten una comunicación más clara entre miembros del equipo.
- Reducen la necesidad de explicaciones verbales o documentación externa.
- Garantizar el mantenimiento a largo plazo
- Ayudan a recordar el propósito de ciertas implementaciones tiempo después.
- Simplifican la depuración y actualización del código.

Tipos de comentarios:

- `//`: Comentario de una sola línea
- `/* */`: Comentario de múltiples líneas
- `///`: Comentario de documentación

Lección 2: Variables y Tipos de Datos

Excelente has logrado crear tu primer programa en dart y no fue muy difícil ahora veamos lo que son las variables y tipos de datos. Dart es un lenguaje de tipado estático, lo que significa que debes declarar el tipo de una variable antes de usarla.

```
void main(){  
    // Variables  
  
    String nombre = 'Juan'; // Tipo de dato cadena  
    int edad = 25; // Tipo de dato entero  
    double altura = 1.70; // Tipo de dato double  
    bool esCliente = true; // Tipo de dato booleano  
  
    // Imprimiendo variables  
  
    print(nombre);  
    print(edad);  
    print(altura);  
    print(esCliente);  
}
```

void main(): Como en el ejemplo anterior, esta línea define la función principal del programa. Aquí, la función principal no toma ningún argumento y no devuelve ningún valor (**void**).

Variables:

String nombre = 'Juan'; Declara una variable llamada **nombre** de tipo **String** (cadena de texto) y le asigna el valor 'Juan'.

int edad = 25; Declara una variable llamada **edad** de tipo **int** (entero) y le asigna el valor 25.

double altura = 1.70; Declara una variable llamada **altura** de tipo **double** (número de punto flotante) y le asigna el valor 1.70.

bool esCliente = true; Declara una variable llamada **esCliente** de tipo **bool** (booleano) y le asigna el valor **true**.

Imprimir variables:

print(nombre); La variable muestra el valor de **Juan**.

print(edad); La variable muestra el valor de **25**.

print(altura); La variable muestra el valor **1.70**.

print(esCliente); La variable muestra el valor de **true**.

Lección 2: Interpolación en Dart

¿Qué es la interpolación?

La interpolación en Dart se refiere a la capacidad de incluir valores de variables o expresiones dentro de cadenas de texto. La sintaxis principal para la interpolación en Dart es el uso del signo de dólar `$` seguido de la variable o expresión que deseas incluir en la cadena. Existen dos formas principales de interpolación en Dart:

1.- Interpolación de variables en cadenas de texto:

```
void main(){  
    String nombre = 'Juan';  
    int edad = 25;  
  
    // Interpolacion de variables en una cadena de texto  
    String mensaje = 'Hola, mi nombre es $nombre y tengo $edad años';  
    print(mensaje);  
}
```

En este ejemplo, las variables `nombre` y `edad` se incorporan en la cadena de texto utilizando `$nombre` y `$edad`. Al imprimir `mensaje`, se mostrará la cadena completa con los valores actuales de las variables.

```
● PS C:\Users\Joshu\Documents\DartCurso\dart_basics\bin\Leccion1> dart .\interpolacion.dart  
Hola, mi nombre es Juan y tengo 25 años
```

2.- Interpolación de expresiones en cadenas de texto:

```
void main(){  
    int a = 5;  
    int b = 7;  
  
    // Interpolacion de expresiones en una cadena de texto  
    String resultado = 'La suma de $a y $b es ${a + b}';  
    print(resultado);  
}
```

Aquí, se utiliza `${a + b}` para incluir el resultado de la expresión `a + b` dentro de la cadena. Puedes utilizar llaves `{}` para delimitar expresiones más complejas.

Resultado:

```
PS C:\Users\Joshu\Documents\DartCurso\dart_basics\bin\Leccion1> dart .\interpolacion2.dart  
La suma de 5 y 7 es 12
```

La interpolación en Dart facilita la construcción de cadenas de texto dinámicas y más legibles al evitar la concatenación manual de strings y valores. Además, permite incluir expresiones complejas directamente en el contexto de una cadena.

Lección 3: Inferencia de Tipos var

Este ejemplo muestra cómo Dart puede inferir automáticamente el tipo de una variable al asignarle un valor, gracias a la palabra clave var

```
void main(){  
  // Dart puede inferir el tipo automaticamente  
  
  var nombre = 'Maria'; // Dart sabe que es tipo String  
  var edad = 30;        // Dart sabe que es tipo int  
  var altura = 1.90;    // Dart sabe que es tipo  
  var esMaestro = true; // Dart sabe que es tipo bool  
  
  print('$nombre tiene $edad años');  
}
```

var: Permite que Dart deduzca el tipo de la variable en tiempo de compilación basándose en el valor asignado.

```
PS C:\Users\Joshu\Documents\DartCurso\dart_basics\bin\Leccion1> dart .\Inferencia_de_Tipos_con_var.dart  
Maria tiene 30 años
```

- 'María' → String.
- 30 → int.
- 1.68 → double.
- false → bool.

Ventajas de la Inferencia

Código más limpio: No necesitas escribir explícitamente el tipo (ej: **String nombre = 'María'**).

Seguridad de tipos: Aunque usas var, Dart sigue siendo fuertemente tipado.

Lección 4: Constantes

¿Qué son las Constantes en Dart?

En Dart, las constantes son variables cuyo valor no puede cambiar después de su asignación. Se utilizan para definir valores fijos que deben mantenerse invariables durante la ejecución del programa. Dart ofrece dos formas de declarar constantes:

- **final**
- **const**

Ambas garantizan inmutabilidad, pero con diferencias clave en su comportamiento y uso.

Constantes con final

Características:

- **Asignación única:** El valor se asigna una sola vez (en tiempo de ejecución).
- **Flexibilidad:** Puede inicializarse con valores calculados en runtime (ej: resultados de funciones, inputs de usuario).

Uso común:

- Datos que se determinan al ejecutar el programa (ej: hora actual, respuesta de una API).
- Objetos que no deben reasignarse, pero cuyos datos internos pueden modificarse (ej: una lista final).

```
void main() {  
  // final: Se asigna una sola vez en tiempo de ejecución  
  final DateTime ahora = DateTime.now();  
  
  print('Hora actual: $ahora');  
}
```


Constantes con const

Características:

- Asignación en compilación: El valor debe conocerse antes de ejecutar el programa (en tiempo de compilación).
- Optimización: Dart almacena estas constantes en memoria de forma eficiente.

Inmutabilidad estricta:

- El valor debe ser un literal (ej: 3.1416, 'Hola') o una expresión constante (ej: 2 + 3).
- Los objetos const son completamente inmutables (no se pueden modificar sus propiedades).

Lección 5: Ingresar datos en dart

En Dart, puedes ingresar datos desde el usuario utilizando la biblioteca `dart:io` y la función `stdin.readLineSync()`

```
// Importamos la biblioteca dart:io
import 'dart:io';

void main(){

  //Solicitamos la entrada del usuario:
  print('Ingrese su nombre: ');

  // Utilizamos la funcion stdin.readLineSync() para leer una linea de entrada del usuario:
  String nombre = stdin.readLineSync()!;

  // Muestra en la consola el nombre ingresado por el usuario:
  print('Su nombre es: $nombre');
}
```

Antes de utilizar la función `stdin.readLineSync()`, necesitas importar la biblioteca `dart:io`. Esta biblioteca proporciona funciones de entrada/salida estándar.

Utilizamos la función `print()` para mostrar un mensaje que solicite la entrada al usuario.

Utilizamos la función `stdin.readLineSync()` para leer una línea de entrada del usuario. Esta función devuelve una cadena de texto que puedes convertir al tipo de dato deseado.

Aquí, `nombre` contendrá la cadena de texto ingresada por el usuario. El `!` al final se utiliza para indicar al compilador que estamos seguros de que la función `readLineSync()` nunca devolverá un valor nulo.

Al final se muestra lo que el usuario ha ingresado.

```
PS C:\Users\ala_m\Documents\dart\programas> dart .\ingresarDatos.dart
Ingrese su nombre:
Juan
Su nombre es: Juan
PS C:\Users\ala_m\Documents\dart\programas>
```

Como podemos observar al ingresar el nombre este sale debajo en vez de estar al lado donde dice “Ingrese su nombre:”.

Veamos cómo podemos solucionar esto en dart

Para lograr que el nombre del usuario se imprima en la misma línea donde se solicitó, puedes utilizar la función `stdout.write()` en lugar de `print()` para la solicitud del nombre.

```
// Importamos la biblioteca dart:io
import 'dart:io';

void main(){

    // Solicitar entrada del usuario sin imprimir nueva linea automaticamente
    stdout.write('Ingrese su nombre: ');

    // Leer la entrada del usuario
    String nombre = stdin.readLineSync()!;

    // Imprime el nombre ingresado por el usuario
    print('Su nombre es: $nombre');

}
```

En este ejemplo, `stdout.write('Ingrese su nombre: ');` solicita el nombre del usuario sin agregar automáticamente una nueva línea al final.

Luego, `stdin.readLineSync()` lee la entrada del usuario, y `print('Su nombre es: $nombre');` imprime el saludo junto con el nombre ingresado.

Al usar `stdout.write()`, puedes controlar dónde aparece la entrada del usuario y la salida en la consola, manteniendo la solicitud y la respuesta en la misma línea.

El resultado final sería el siguiente:

```
PS C:\Users\ala_m\Documents\dart\programas> dart .\ingresarDatos.dart
Ingrese su nombre: Juan
Su nombre es: Juan
PS C:\Users\ala_m\Documents\dart\programas>
```

Ahora haremos lo mismo pero utilizando números.

Qué te parece hacer un programa el cual solicita al usuario que ingrese dos números y luego imprime la suma de esos números.

```
import 'dart:io';

void main(){

    // Solicitar entrada del primer numero sin imprimir nueva linea automaticamente
    stdout.write('Ingrese el primer numero: ');

    // Leer la entrada del usuario y convertir a un numero entero
    String entradaNumero1 = stdin.readLineSync();
    int numero1 = int.parse(entradaNumero1);

    // Solicitar entrada del segundo numero sin imprimir nueva linea automaticamente
    stdout.write('Ingrese el segundo numero: ');

    // Leer la entrada del usuario y convertir a un numero entero
    String entradaNumero2 = stdin.readLineSync();
    int numero2 = int.parse(entradaNumero2);

    // Calcular y mostrar la suma
    int suma = numero1 + numero2;
    print('La suma de $numero1 y $numero2 es: $suma');
}
```

import 'dart:io';: Importamos la biblioteca `dart:io` para tener acceso a las funciones de entrada/salida estándar.

void main(): La función principal del programa. Aquí es donde comienza la ejecución.

stdout.write('Ingrese el primer número: '); Utilizamos **stdout.write()** en lugar de **print()** para solicitar el primer número sin imprimir automáticamente una nueva línea después de la solicitud. Esto permite que el usuario ingrese su respuesta en la misma línea.

Leer y convertir el primer número:

String entradaNumero1 = stdin.readLineSync(!); Leemos la entrada del usuario como una cadena de texto y la almacenamos en **entradaNumero1**.

int numero1 = int.parse(entradaNumero1); Convertimos la cadena **entradaNumero1** a un número entero utilizando **int.parse()** y lo almacenamos en **numero1**.

stdout.write('Ingrese el segundo número: '); Hacemos lo mismo para solicitar el segundo número.

Leer y convertir el segundo número:

String entradaNumero2 = stdin.readLineSync(!); Leemos la entrada del usuario como una cadena de texto y la almacenamos en **entradaNumero2**.

int numero2 = int.parse(entradaNumero2); Convertimos la cadena **entradaNumero2** a un número entero utilizando **int.parse()** y lo almacenamos en **numero2**.

Calcular y mostrar la suma:

int suma = numero1 + numero2; Calculamos la suma de los dos números y la almacenamos en **suma**.

print('La suma de \$numero1 y \$numero2 es: \$suma'); Utilizamos **print()** para mostrar la suma junto con los números ingresados por el usuario.

El resultado sería el siguiente:

```
PS C:\Users\ala_m\Documents\dart\programas> dart .\sumaDeDosNumeros.dart
Ingrese el primer numero: 5
Ingrese el segundo numero: 5
La suma de 5 y 5 es: 10
PS C:\Users\ala_m\Documents\dart\programas>
```

Ejercicios

“La práctica constante es el camino hacia la maestría. Cada ejercicio es un paso hacia el dominio, una oportunidad para aprender y mejorar. En el mundo del conocimiento, el esfuerzo diario es la llave que desbloquea puertas y revela nuevas capacidades. No temas desafiar tus límites, porque en cada ejercicio reside el potencial para alcanzar nuevas alturas. ¡Practica con pasión, aprende con determinación y conviértete en el maestro de tu propio camino!”

Recuerda que la práctica regular no solo mejora tus habilidades técnicas, sino que también fortalece tu perseverancia, paciencia y confianza. ¡Adelante, cada ejercicio cuenta!

Ejercicio 1: Calculadora de Índice de Masa Corporal (IMC)

En este ejercicio, solicitamos al usuario su peso y altura, calculamos el IMC y mostramos el resultado utilizando la interpolación.

```
import 'dart:io';

void main(){

    // Solicitar al usuario que ingrese su peso en kilogramos
    stdout.write('Ingrese su peso en kilogramos(Kg): ');

    String entradaPeso = stdin.readLineSync!;
    double peso = double.parse(entradaPeso);

    // Solicitar al usuario que ingrese su altura en metros
    stdout.write('Ingrese su altura en metros(m): ');

    String entradaAltura = stdin.readLineSync!;
    double altura = double.parse(entradaAltura);

    // Calcular el Índice de Masa Corporal (IMC)
    double imc = peso / (altura * altura);

    // Mostrar el resultado con interpolacion
    print('Su Índice de Masa Corporal (IMC) es: $imc');
}
```

Ejercicio 2: Saludo Personalizado

En este ejercicio, solicitamos al usuario su nombre y género, y luego creamos un saludo personalizado utilizando la interpolación basada en el género ingresado.

```
import 'dart:io';

void main(){

    // Solicitar al usuario que ingrese su nombre
    stdout.write('Ingrese su nombre: ');
    String nombre = stdin.readLineSync()!;

    // Solicitar al usuario que ingrese su genero
    stdout.write('Ingrese su genero (Masculino/Femenino): ');
    String genero = stdin.readLineSync()!;

    // Crear un saludo personalizado utilizado interpolacion
    String saludo = genero.toLowerCase() == 'masculino'
        ? 'Hola, Sr. $nombre'
        : 'Hola, Sra. $nombre';

    // Mostrar el saludo
    print(saludo);
}
```

En este ejercicio(Ejercicio 2) utilizo una condicional el cual voy a explicar para que se pueda entender.

```
String saludo = genero.toLowerCase() == 'masculino'
    ? 'Hola, Sr. $nombre'
    : 'Hola, Sra. $nombre';
```

Esta línea de código utiliza una expresión condicional (también conocida como operador ternario) para determinar el saludo personalizado basado en el género ingresado por el usuario. Analicemos cada parte de esta expresión:

genero.toLowerCase() == 'masculino': La función `toLowerCase()` se utiliza para convertir la cadena `genero` a minúsculas, asegurándonos de manejar entradas en diferentes formatos (mayúsculas, minúsculas, mixtas, etc.). Luego, se compara si la cadena resultante es igual a la cadena `'masculino'`. Esta parte de la expresión evalúa a `true` si el género es masculino y `false` en caso contrario.

Operador ternario (? :): El operador ternario se utiliza para tomar decisiones en una sola línea. La forma general es `condición ? expresiónSiTrue : expresiónSiFalse`. En este caso:

Si la condición es verdadera (`genero.toLowerCase() == 'masculino'`), entonces la expresión devuelve `'Hola, Sr. $nombre'`.

Si la condición es falsa, entonces la expresión devuelve `'Hola, Sra. $nombre'`.

Resultado asignado a `saludo`: La cadena resultante de la expresión condicional se asigna a la variable `saludo`. Por lo tanto, `saludo` contendrá el saludo personalizado basado en el género ingresado por el usuario.

En resumen, esta expresión condicional permite crear un mensaje de saludo personalizado dependiendo del género ingresado por el usuario. Si el género es "masculino", el saludo será '`Hola, Sr. $nombre`'; si es cualquier otra cosa, será '`Hola, Sra. $nombre`'. La interpolación se utiliza para incluir el nombre del usuario en el mensaje de saludo.

Lección 5: Estructuras de Control de Flujo

Estructuras de control condicionales **if** simple

```
void main(){  
    int edad = 18;  
    if (edad >= 18);  
        print('Eres mayor de edad');  
}
```

Aquí estamos usando la palabra clave **if** seguida de una condición entre paréntesis. La condición es `edad >= 18`, que evalúa si la variable `edad` es mayor o igual a 18.

Si la condición del **if** es verdadera, se ejecutará el bloque de código entre las llaves `{}`. En este caso, imprimirá en la consola el mensaje `Eres mayor de edad`.

Cada bloque de código dentro del **if** (y otras estructuras de control) debe cerrarse con llaves `}`.

En resumen, este programa simple verifica si la variable `edad` es mayor o igual a 18. Si es así, imprime un mensaje de bienvenida. Si la condición es falsa, no se ejecuta nada dentro del bloque **if**. La estructura **if** es fundamental para tomar decisiones en un programa basándose en condiciones específicas.

if y **else**

La estructura de `if` y `else` se utiliza para manejar casos alternativos.

```
import 'dart:io';

void main(){

  stdout.write('Digite su edad: ');

  String inputEdad = stdin.readLineSync!;
  int edad = int.parse(inputEdad);

  if (edad >= 18){

    print('Eres mayor de edad. ¡Bienvenido!');

  } else {

    print('Eres menor de edad. Acceso restringido.');

  }

}
```

Aquí, el programa imprimirá un mensaje diferente dependiendo de si la variable `edad` es mayor o igual a 18.

Básicamente el bloque `else` es esencial para tomar decisiones en tu código. Permite que tu programa maneje casos en los que la condición principal no es verdadera y proporciona una ruta alternativa de ejecución.

`else if`

La estructura de `else if` para manejar múltiples condiciones.

```
import 'dart:io';

void main() {
  stdout.write('Digite su edad: ');

  String inputEdad = stdin.readLineSync();
  int edad = int.parse(inputEdad);

  if (edad >= 18) {
    print('Eres mayor de edad ¡Bienvenido!');
  } else if (edad >= 13) {
    print('Eres un adolescente. Acceso parcial');
  } else {
    print('Eres un niño. Acceso restringido');
  }
}
```

La estructura `else if` se utiliza para evaluar múltiples condiciones en orden y ejecutar el bloque de código asociado al primer `else if` que tenga una condición verdadera.

Orden Importa: Las condiciones se evalúan en orden. Si una condición es verdadera y su bloque de código se ejecuta, las condiciones posteriores no se evalúan.

Independencia de Bloques: Cada bloque `else if` es independiente. Si una condición se cumple y su bloque de código se ejecuta, los bloques `else if` asociados no se evalúan.

Último `else`: El último bloque `else` (si está presente) se ejecuta si ninguna de las condiciones anteriores es verdadera.

switch

La estructura `switch` para manejar múltiples casos

```

import 'dart:io';

void main(){
    stdout.write('Digite el día de la semana: ');
    String dia = stdin.readLineSync()!;

    String diaSemana = dia.toLowerCase();

    switch(diaSemana){
        case 'lunes':
            print('Comienza la semana');
            break;

        case 'martes':
            print('Casi a mitad de semana');
            break;

        case 'miercoles':
            print('Mitad de semana');
            break;

        case 'jueves':
            print('Casi viernes!!!');
            break;

        case 'viernes':
            print('!Viernes al fin!');
            break;

        default:
            print('Otro día de la semana');
    }
}

```

`switch` se utiliza cuando se quiere comparar una variable con varios valores posibles. En este ejemplo, se imprime un mensaje diferente dependiendo del valor de `diaSemana`.

Recuerda siempre colocar `break` al final de cada `case` para evitar la ejecución continua de los siguientes bloques.

La entrada del usuario se convierte a minúsculas usando `toLowerCase()`. Esto asegura que las comparaciones en el `switch` sean insensibles a mayúsculas y minúsculas.

Ahora hagamos 3 ejercicios de lo aprendido.

Ejercicios:

Ejercicio 1: Verificación de Rango

Escribe un programa que solicite al usuario ingresar un número e imprima en qué rango se encuentra el número:

Si el número es menor que 0, imprime "Número negativo".

Si el número es mayor que 0 pero menor o igual a 10, imprime "Número en el rango [1-10]".

Si el número es mayor que 10, imprime "Número mayor que 10".

```
import 'dart:io';

void main(){

  stdout.write('Digite un numero del (1-10): ');

  String inputNumero = stdin.readLineSync()!;
  int numero = int.parse(inputNumero);

  if (numero < 0){

    print('El numero $numero es numero negativo');

  } else if ( numero > 0 && numero <= 10){

    print('El numero $numero esta en el rango de [1-10]');

  } else if (numero > 10 ){

    print('El numero $numero es mayor que 10');

  } else {

    print('Error');

  }

}
```

Ejercicio 2: Día de la Semana

Escribe un programa que solicite al usuario ingresar un número del 1 al 7 e imprima el día de la semana correspondiente:

- Usa una estructura `if-else` para imprimir el nombre del día.
- Si el usuario ingresa un número fuera del rango, imprime "Número no válido".

```
import 'dart:io';

void main(){

  print('Digite un numero del 1 al 7 para imprimir la semana correspondiente');

  stdout.write('\nDigite un numero del (1 al 7): ');

  String inputSemana = stdin.readLineSync!;
  int semana = int.parse(inputSemana);

  if (semana == 1){
    print('Lunes');
  } else if (semana == 2){
    print('Martes');
  } else if (semana == 3){
    print('Miercoles');
  } else if (semana == 4){
    print('Jueves');
  } else if (semana == 5){
    print('Viernes');
  } else if (semana == 6){
    print('Sabado');
  } else if (semana == 7){
    print('Domingo');
  } else {
    print('Numero no valido');
  }
}
```

Ejercicio 3: Conversión de Calificación

Escribe un programa que solicite al usuario ingresar una calificación (A, B, C, D, F) y utilice una estructura `switch` para imprimir un mensaje correspondiente:

- A: "Excelente trabajo".
- B: "Buen trabajo".
- C: "Trabajo promedio".
- D: "Trabajo insatisfactorio".
- F: "Has fallado".
- Para cualquier otra calificación, imprime "Calificación no válida".

```
import 'dart:io';

void main(){

  stdout.write('Ingrese una calificacion(A,B,C,D,F): ');

  String calificacion = stdin.readLineSync()!.toLowerCase();

  switch(calificacion){

    case 'a':
      print('Excelente trabajo');
      break;

    case 'b':
      print('Buen trabajo');
      break;

    case 'c':
      print('Trabajo promedio');
      break;

    case 'd':
      print('Trabajo insatisfactorio');
      break;

    case 'f':
      print('Has fallado');
      break;

    default:
      print('Calificacion no valida');

  }

}
```

Aquí el uso de `toLowerCase()` garantiza que el programa sea insensible a mayúsculas y minúsculas. De esta manera, el usuario puede ingresar 'A' o 'a', y ambos serán tratados de la misma manera en la comparación del bloque `switch`. Esto mejora la usabilidad del programa y evita

problemas cuando los usuarios ingresan calificaciones con mayúsculas o minúsculas. Siempre es una buena práctica normalizar la entrada de usuario de esta manera cuando la distinción entre mayúsculas y minúsculas no es relevante para el propósito del programa.

Lección 6: Estructuras de control iterativas

Las estructuras de control iterativas, también conocidas como bucles, permiten repetir un bloque de código varias veces según una condición dada. En Dart, existen principalmente dos tipos de bucles: `for` y `while`.

Bucle `for`:

El bucle `for` es útil cuando sabes de antemano cuántas veces deseas repetir un bloque de código. Su sintaxis básica es:

```
for (inicialización; condición; incremento/decremento) {  
    // Código a repetir  
}
```

Aquí hay un ejemplo simple de un bucle `for` que imprime los números del 1 al 5:

```
void main(){  
    for (int i = 1; i <= 5; i++){  
        print(i);  
    }  
}
```

En este ejemplo:

`int i = 1`: Inicializa la variable de control `i` en 1.

`i <= 5`: La condición del bucle. Se repetirá mientras `i` sea menor o igual a 5.

`i++`: Incrementa `i` en 1 después de cada iteración.

De igual forma se puede hacer del 1 al 10:

```
void main(){  
    for(int i = 1; i <= 10; i++){  
        print(i);  
    }  
}
```

Imprimir números pares del 2 al 20

```
void main(){  
    for (int i = 2; i <= 20; i += 2){  
        print(i);  
    }  
}
```

void main(): Esta es la función principal del programa. En Dart, `main()` es el punto de entrada de la ejecución del programa.

for (int i = 2; i <= 20; i += 2) { ... }: Este es un bucle `for`. Aquí hay un desglose de sus partes:

`int i = 2:` Inicialización. `i` se establece en 2 al principio del bucle.

`i <= 20:` Condición. El bucle continuará mientras `i` sea menor o igual a 20.

`i += 2:` Incremento. `i` se incrementa en 2 después de cada iteración.

print(i); Dentro del bucle, se imprime el valor actual de `i` en cada iteración.

Ahora, veamos cómo se ejecutará este bucle:

- En la primera iteración, `i` es 2, y se imprime.
- En la segunda iteración, `i` se incrementa en 2 y se convierte en 4, y se imprime.
- Este proceso continúa hasta que `i` llega a 20.

El resultado de este bucle será imprimir los números pares del 2 al 20, ya que `i` comienza en 2, se incrementa en 2 en cada iteración y la condición es que `i` sea menor o igual a 20.

Contar hacia atrás desde 10

```
void main(){
    for(int i = 10; i >= 0; i--){
        print(i);
    }
}
```

void main(): Esta es la función principal del programa. En Dart, `main()` es el punto de entrada de la ejecución del programa.

for (int i = 10; i >= 0; i--) { ... }: Este es un bucle `for`. Aquí hay un desglose de sus partes:

`int i = 10`: Inicialización. `i` se establece en 10 al principio del bucle.

`i >= 0`: Condición. El bucle continuará mientras `i` sea mayor o igual a 0.

`i--`: Decremento. `i` se reduce en 1 después de cada iteración.

print(i); Dentro del bucle, se imprime el valor actual de `i` en cada iteración.

Ahora, veamos cómo se ejecutará este bucle:

- En la primera iteración, `i` es 10, y se imprime.
- En la segunda iteración, `i` se reduce a 9, y se imprime.
- Este proceso continúa hasta que `i` llega a 0.

El resultado de este bucle será imprimir los números del 10 al 0 en orden descendente, ya que `i` comienza en 10, se decrementa en 1 en cada iteración y la condición es que `i` sea mayor o igual a 0

